

FINTRACK

Personal Finance Tracker

A Command-Line Python Project

Project Report submitted for

CSE1021 - Introduction to Problem Solving and Programming

Astitva Srivastava

Registration No: 26BAS10101

VIT Bhopal University

GitHub: github.com/astitva0812/FinTrack

Table of Contents

Section	Title
1	Introduction
2	Problem Statement
3	Objectives and Scope
4	Functional Requirements
5	Non-Functional Requirements
6	System Architecture and Workflow
7	Data Design
8	Design Decisions and Rationale
9	Implementation Details
10	Algorithms Used
11	Screenshots and Results
12	Testing Approach
13	Challenges Faced and Resolutions
14	Learnings and Key Takeaways
15	Future Enhancements
16	Conclusion
17	References

1. Introduction

FinTrack is a command-line personal finance tracker developed in Python. The project was created to apply the problem-solving, programming, data-structure, and algorithmic concepts covered in CSE1021 to a practical problem.

The application focuses on a small and manageable workflow: a user enters a name, records income or expense transactions, stores them locally, and performs basic analysis on the stored data. The design deliberately remains procedural and uses Python's standard library rather than introducing frameworks, classes, databases, or other technologies outside the current project scope.

The project demonstrates how concepts such as loops, conditional statements, lists, dictionaries, sets, summation, maximum finding, reversal, duplicate removal, partitioning, and selection-based ordering can be combined into one working application.

2. Problem Statement

Individuals can lose track of day-to-day spending when transactions are recorded manually or not recorded consistently. FinTrack addresses this problem with a lightweight terminal-based application that allows transactions to be entered quickly and provides immediate access to stored transactions and basic financial calculations.

The project is intentionally educational: its main purpose is to demonstrate fundamental programming and algorithmic techniques while solving a small real-world problem.

3. Objectives and Scope

Objectives

- Record personal income and expense transactions through a command-line interface.
- Apply fundamental problem-solving and algorithmic techniques from CSE1021.
- Use lists, dictionaries, and sets to represent and process transaction data.
- Calculate totals, net balance, highest expense, unique categories, and the Kth smallest expense.
- Persist transaction data using a local JSON file.
- Demonstrate the connection between syllabus concepts and a practical application.

Scope

- Adding and viewing transactions.
- Basic transaction analysis and list processing.
- User-specific transaction storage within the local JSON file.
- Input validation and basic error handling.
- No graphical interface, online synchronization, bank integration, or advanced financial forecasting.

4. Functional Requirements

ID	Requirement
FR1	The system shall ask for a username and maintain that user's transactions separately from other usernames stored in the same data file.
FR2	The system shall allow a user to add a transaction containing a category, amount, and type (income or expense), and validate the amount and
FR3	The system shall display all transactions recorded for the current user.

ID	Requirement
FR4	The system shall find the highest expense while excluding income entries from the comparison.
FR5	The system shall display transactions in reverse order.
FR6	The system shall remove duplicate transaction entries using category, amount, and type together.
FR7	The system shall separate transactions into income and expense groups.
FR8	The system shall calculate total income, total expenses, and net balance.
FR9	The system shall report transaction counts, income/expense counts, and distinct categories.
FR10	The system shall determine the Kth smallest expense for a valid user-specified K.
FR11	The system shall load and save transaction data using data.json.

5. Non-Functional Requirements

ID	Requirement
NFR1 - Usability	A simple numbered command-line menu should allow the user to operate the program without additional software beyond what is installed on the user's machine.
NFR2 - Reliability	Transaction changes are saved to the JSON file when a transaction is added, reducing the risk of losing previously saved data.
NFR3 - Data Persistence	Stored transactions remain available across program executions through data.json.
NFR4 - Error Handling	Missing or invalid JSON data, invalid amounts, invalid transaction types, invalid menu choices, and invalid K values are handled gracefully.
NFR5 - Resource Efficiency	The program uses Python's standard library and in-memory lists/dictionaries/sets rather than requiring external packages.
NFR6 - Maintainability	The program uses straightforward procedural control flow and clearly named data fields so that individual menu operations are easy to understand and modify.

6. System Architecture and Workflow

The complete implementation is contained in main.py. Although the program is a single-file procedural application, its responsibilities can be described in three logical layers: user interaction, transaction/analysis processing, and persistent storage.

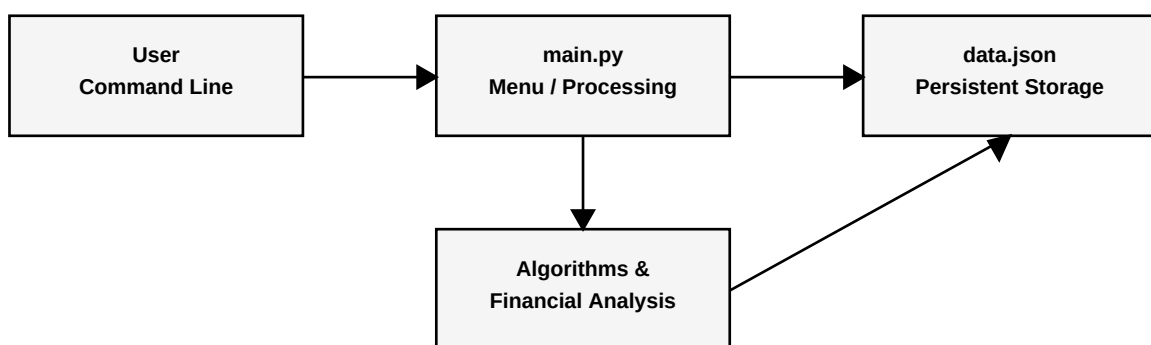


Figure 1. Logical architecture of FinTrack.

Workflow

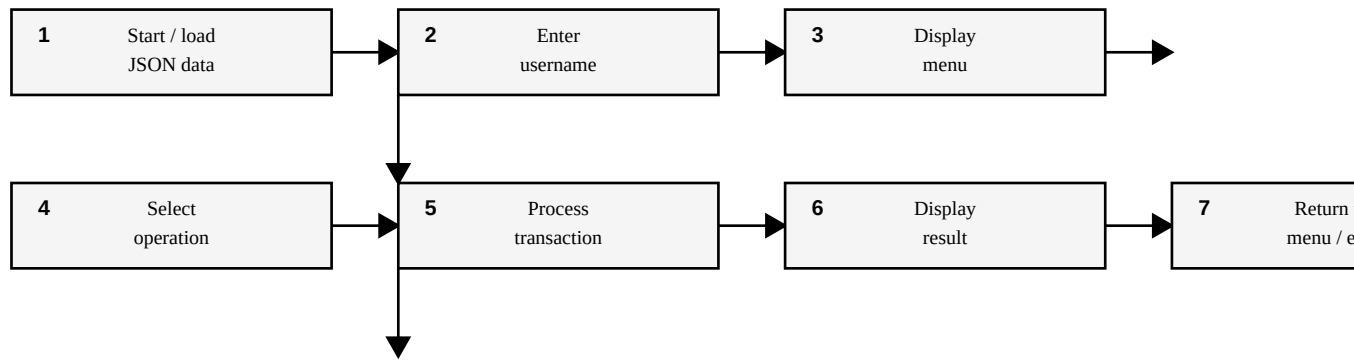


Figure 2. Simplified execution workflow.

Main menu

Option	Operation	Purpose
1	Add transaction	Enter and save a new transaction.
2	View all transactions	Display the current user's transactions.
3	Find highest expense	Find the largest expense amount.
4	Reverse transactions	Display transactions in reverse order.
5	Remove duplicate transactions	Remove repeated transaction entries.
6	Split income vs. expense	Partition transactions by type.
7	Show total income, expense and balance	Calculate financial totals.
8	Transaction statistics	Count transactions and unique categories.
9	Kth smallest expense	Find the requested ranked expense.
10	Exit	End the program.

7. Data Design

FinTrack uses JSON as a lightweight persistent storage format. The top-level structure is a dictionary whose keys are usernames and whose values are lists of transaction dictionaries.

```

{"Astitva": [{"category": "Food", "amount": 300, "type": "expense"}, {"category": "Salary",
"amount": 20000, "type": "income"}]}

```

Each transaction contains three core fields: **category**, **amount**, and **type**. This representation supports traversal, filtering, counting, reversal, duplicate comparison, and financial calculations.

8. Design Decisions and Rationale

User-specific storage. The initial implementation used a shared transaction list. Testing showed that entering another user's name could still expose previously stored transactions. The storage model was therefore changed so each username maps to its own transaction list.

Explicit algorithms. Built-in operations such as `max()` and `sorted()` could shorten some features, but explicit loops and a manual selection-sort-style process were retained so that the underlying CSE1021 algorithms

remain visible in the implementation.

Stable file location. The JSON path is derived from the script's own directory rather than depending on the current working directory. This avoids different save locations when the program is launched from different environments.

JSON instead of a database. JSON provides structured persistent storage without introducing a database system that is outside the project's current scope.

Procedural design. No user-defined classes are used. The implementation stays within the basic Python programming style appropriate to the course scope.

Validation. Transaction type input is restricted to income or expense, and duplicate comparison considers category, amount, and type together so that genuinely different transactions are not incorrectly removed.

9. Implementation Details

The implementation is contained in main.py, with main_backup.py retained as a backup. It imports Python's standard-library json and os modules. The main program loads data.json, obtains the current user's transaction list, repeatedly displays the menu, processes the selected operation, and saves changes where required.

Menu option	Primary CSE1021 concept
1 - Add	Dictionaries, input/output, validation, JSON storage
2 - View all	List traversal and iteration
3 - Highest expense	Maximum finding
4 - Reverse view	Array/list reversal
5 - Remove duplicates	Duplicate removal with nested comparison
6 - Split income/expense	Partitioning
7 - Totals	Summation
8 - Counts and categories	Counting and set operations
9 - Kth smallest	Selection-sort-style ordering and indexing

10. Algorithms Used

Summation: the program iterates through transactions and accumulates income and expense totals.

```
total_income = 0
for transaction in transactions:
    if transaction['type'] == 'income':
        total_income += float(transaction['amount'])
```

Maximum finding: the program first isolates expense transactions and then compares each amount with the current highest expense.

```
exp_only = []
for t in tx:
    if t['type'] == 'expense':
        exp_only.append(t)
high = exp_only[0]
for t in exp_only:
    if float(t['amount']) > float(high['amount']):
        high = t
```

Reversal: transactions are rebuilt in reverse order using list insertion at the beginning.

Duplicate removal: transaction fields are compared so that repeated category/amount/type combinations can be removed.

Partitioning: transactions are separated into income and expense collections according to the type field.

Counting and sets: loops count total and type-specific transactions, while a set is used to identify distinct categories.

Kth smallest: expense values are copied and ordered using a manual selection-sort-style algorithm before selecting the requested rank.

11. Screenshots and Results

The report retains the execution evidence supplied with the original report. The figures demonstrate transaction entry, financial totals, separation of users, duplicate removal, and Git-based repository work.

8. Screenshots / Results

The following screenshots were captured while verifying that transaction data for different users remains properly separated, following the identification and correction of the cross-user data issue described in Section 10.

```
C:\Program Files\WindowsP... x + -
Enter your name: Test1
Welcome, Test1 - let's track your finances!

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 1
Category (e.g. Food, Rent, Salary): Salary
Amount: 20000
Type (income/expense): income
Added and saved!

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 1
Category (e.g. Food, Rent, Salary): Rent
Amount: 2000
Type (income/expense): expense
Added and saved!

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
```

Figure 1 - User Test1 adds a Salary and a Rent transaction; both are confirmed saved.

```
C:\Program Files\WindowsP... x + -
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 1
Category (e.g. Food, Rent, Salary): food
Amount: 8000
Type (income/expense): expense
Added and saved!

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 7
Total income: 20000.0
Total expense: 8000.0
Net balance: 12000.0

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: |
```

Figure 2 - User Test1 adds a Food expense, then views the totals; income, expense, and net balance are calculated correctly.

Figure 3. Supplied execution evidence from the original report.

```
C:\Program Files\WindowsP...
Enter your name: Test2
Welcome, Test2 - let's track your finances!

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 2

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 1
Category (e.g. Food, Rent, Salary): Rent
Amount: 30000
Type (income/expense): expense
Added and saved!

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
```

Figure 3 - A different user, Test2, views their transactions and sees an empty list, confirming that Test1's data is not accessible to them.

```
C:\Program Files\WindowsP...
Amount: 30000
Type (income/expense): expense
Added and saved!

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 2
Rent - 30000 - expense

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: 5
Duplicates removed. Remaining transactions: 1

1. Add a transaction
2. View all transactions
3. Find highest expense
4. View transactions (newest first)
5. Remove duplicate transactions
6. Split into income vs. expense
7. Show total income, expense, and balance
8. Exit
Enter your choice: |
```

Figure 4 - User Test2 adds a Rent expense, views it, and then successfully removes a duplicate entry.

Figure 4. Supplied execution evidence from the original report.

Corrupted save file. An empty data.json file caused a JSONDecodeError in an earlier version. JSON decoding errors are now handled so that the application can start with an empty data set.

Cross-user data visibility. An initial shared transaction list caused data entered by one username to remain visible to another. The storage model was changed to map each username to its own list of transactions.

Transaction-type validation. An earlier typing error could result in an invalid type being saved. Validation was tightened so only income or expense is accepted.

14. Learnings and Key Takeaways

- Individual algorithms such as summation, maximum finding, reversal, duplicate removal, and partitioning become more useful when combined in one practical program.
- A program can execute without crashing and still produce an incorrect result; manual verification of expected output is therefore important.
- Testing different launch methods can reveal file-path problems that are not visible during a single execution method.
- Data representation matters: organizing transactions by username solved a cross-user data-separation problem.
- Input validation and error handling are part of building a reliable program, even for a small command-line application.

15. Future Enhancements

- Introduce password protection or another authentication mechanism if the project later needs stronger user separation.
- Allow users to export transaction history or financial summaries to a separate text or report file.
- Add date fields and date-based filtering for transactions.
- Add budget limits and simple budget-versus-expense comparisons.
- Provide a graphical interface if a later course or project scope permits it.

16. Conclusion

FinTrack demonstrates how fundamental CSE1021 programming concepts can be combined into a practical personal finance application. The project uses Python control flow, lists, dictionaries, sets, file handling, JSON storage, and basic algorithms to manage and analyse transaction data.

The development process also demonstrated the importance of testing beyond simply checking whether a program runs. Several issues—including incorrect highest-expense selection, inconsistent file locations, corrupted JSON handling, and cross-user data visibility—were identified through testing and addressed in the implementation.

The final system remains intentionally simple and course-aligned while providing enough functionality to demonstrate multiple algorithmic and data-structure techniques.

17. References

- CSE1021 - Introduction to Problem Solving and Programming, VIT Bhopal University course syllabus.
- Python 3 Documentation - json module: <https://docs.python.org/3/library/json.html>
- Python 3 Documentation - os.path module: <https://docs.python.org/3/library/os.path.html>
- RAPTOR Flowchart Tool: <https://raptor.martincsis.gmu.edu/>

- FinTrack GitHub repository: <https://github.com/astitva0812/FinTrack>